Working on this NewsBot assignment was honestly a huge learning experience for me, and looking back at the notebook file, I can see how much work actually went into it. When our team first opened the template, the whole thing felt pretty intimidating. It was basically a giant checklist of eight different modules that we had to build from scratch. The first real hurdle was right at the beginning of the file where the data loading code was. The original code kept stopping the notebook to ask for a manual file upload from Google Colab, which totally broke the automation. Fixing that by writing a clean, non-blocking script using the Kaggle API token string was a major win for us because it allowed the whole notebook to run straight through without getting stuck. Sampling exactly 120 articles per category from the BBC News dataset gave us a perfect, manageable chunk of 600 total articles to play with, which kept the processing fast so the AI wouldn't lag out.

Getting into the actual text cleaning and processing was where things got really interesting. The code we wrote for cleaning out HTML tags, links, and special characters made a massive difference when you look at the before-and-after text examples. Once we ran the math formula to score the words, it was cool to see what the AI picked up on. The model showed that specific words like "market" or "firm" basically pointed straight to the Business category, while "film" and "award" belonged to Entertainment. When it came to building the actual classifier in the later cells, we ended up testing three different models. The Naive Bayes model completely blew the others out of the water, hitting over 94% accuracy on our test data. One really tricky thing we ran into with that specific model was that it completely broke whenever it encountered negative numbers. Since our sentiment scores could be negative, we had to add a MinMaxScaler to compress all those extra features between 0 and 1. Seeing that fix actually work and keep the model from crashing was super satisfying.

What I liked most about digging into the code was looking at the actual linguistic patterns the AI found in the data. The part-of-speech and syntax cells printed out some really funny summaries that made total sense. For instance, the data proved that Politics articles use the longest, most complicated sentences, averaging about 26 words each, and they use the most action verbs because politicians are always doing or arguing something. Business articles used way more numbers than anyone else, which makes sense with all the stock market data. My favorite part of the whole file was the advanced entity extraction we added at the end. The AI pulled out over seventeen thousand real-world entities and even mapped out a network showing which ones appear together. It was awesome to see a visual web showing exactly how often words like "UK" and "US" or "Tony Blair" and "Britain" were linked in the same news stories.

Overall, making all these different modules work together into one complete pipeline was tough, but it gave me a ton of confidence in what I can actually build with AI now.